Compte rendu – Analyse d’une authentification HTTP avec Wireshark

1. Objectif du TP

L’objectif de ce travail est d’analyser des trames réseau HTTP à l’aide de Wireshark afin d’identifier comment sont transmis les identifiants lors d’une authentification sur un serveur NAS.
Cette analyse permet de mettre en évidence les risques liés à l’utilisation du protocole HTTP pour l’authentification.

2. Environnement de travail

- Poste client : PC sous Windows
- Outil d’analyse : Wireshark
- Serveur : NAS accessible en HTTP
- Adresse du NAS : http://172.16.14.250:8080
- Navigateur : Google Chrome

3. Connexion au NAS (génération du trafic réseau)

Avant toute analyse, il est nécessaire de générer du trafic réseau.

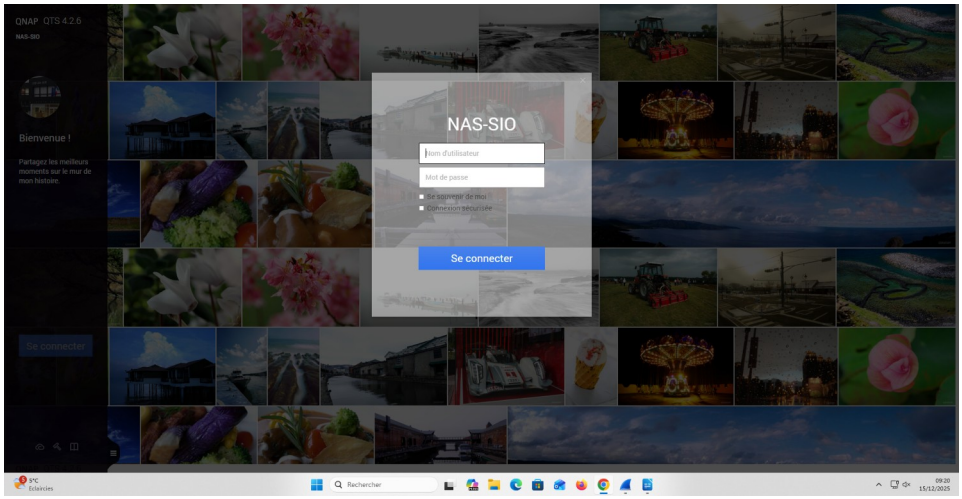
1. Ouvrir un navigateur web.
2. Saisir l’adresse du NAS :

   http://172.16.14.250:8080

3. Accéder à la page de connexion du NAS.
4. Entrer un **nom d’utilisateur** et un **mot de passe**.
5. Valider la connexion.

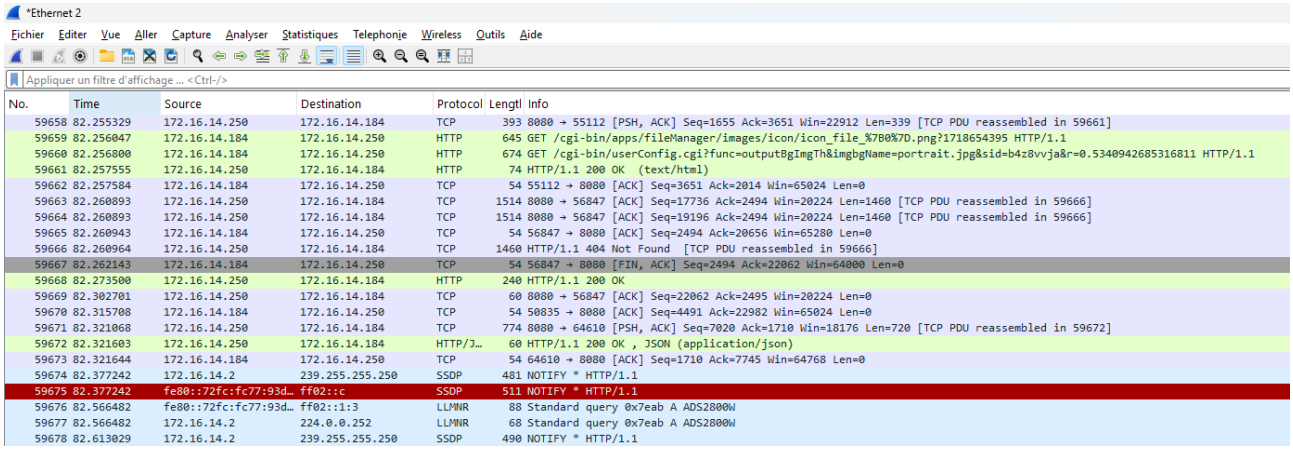
Cette action génère des trames HTTP qui seront ensuite analysées dans Wireshark.

Connexion au NAS via le navigateur



4. Capture du trafic réseau avec Wireshark

1. Lancer Wireshark.
2. Sélectionner l’interface réseau utilisée (Ethernet ou Wi-Fi).
3. Démarrer la capture.
4. Effectuer la connexion au NAS.
5. Arrêter la capture après la connexion réussie.



Interface Wireshark avec la capture en cours

5. Filtrage des trames HTTP POST

L’authentification peut être réalisée via deux méthodes principales :

- **GET**
- **POST**

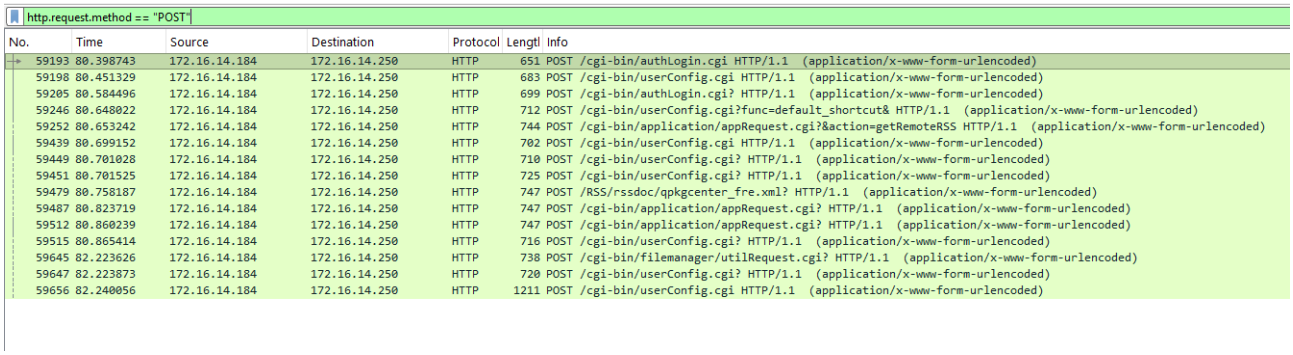
Dans ce cas précis, l’authentification se fait via **POST**.

1. Appliquer le filtre suivant dans Wireshark :

   http.request.method == "POST"

2. Les trames POST apparaissent alors dans la liste.
3. Repérer la trame contenant :

   /cgi-bin/authLogin.cgi



Filtrage des requêtes HTTP POST

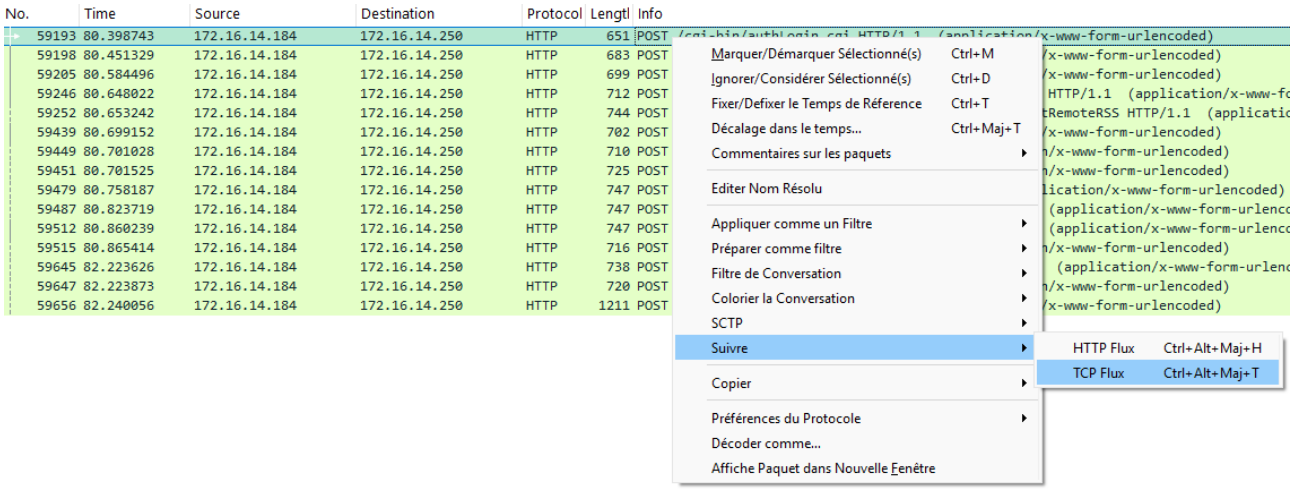
6. Analyse détaillée de la trame d'authentification

1. Faire un **clic droit** sur la trame :

   POST /cgi-bin/authLogin.cgi

2. Sélectionner :

   Follow → TCP Stream



3. Wireshark reconstitue alors l'intégralité de la conversation TCP entre le client et le NAS.

Menu "Follow TCP Stream"

7. Récupération des identifiants

Dans le flux TCP, on observe les paramètres envoyés lors de l'authentification :

user=jed&serviceKey=1&pwd=SlY3ODM3MA%3D%3D&r=0.22534392867493314

- user : identifiant de l'utilisateur
- pwd : mot de passe encodé
- %3D correspond au caractère =

Après décodage URL, on obtient :

SlY3ODM3MA==

8. Décodage du mot de passe (Base64)

Le mot de passe est encodé en **Base64**, ce qui n'est **pas un chiffrement**, mais un simple encodage réversible.

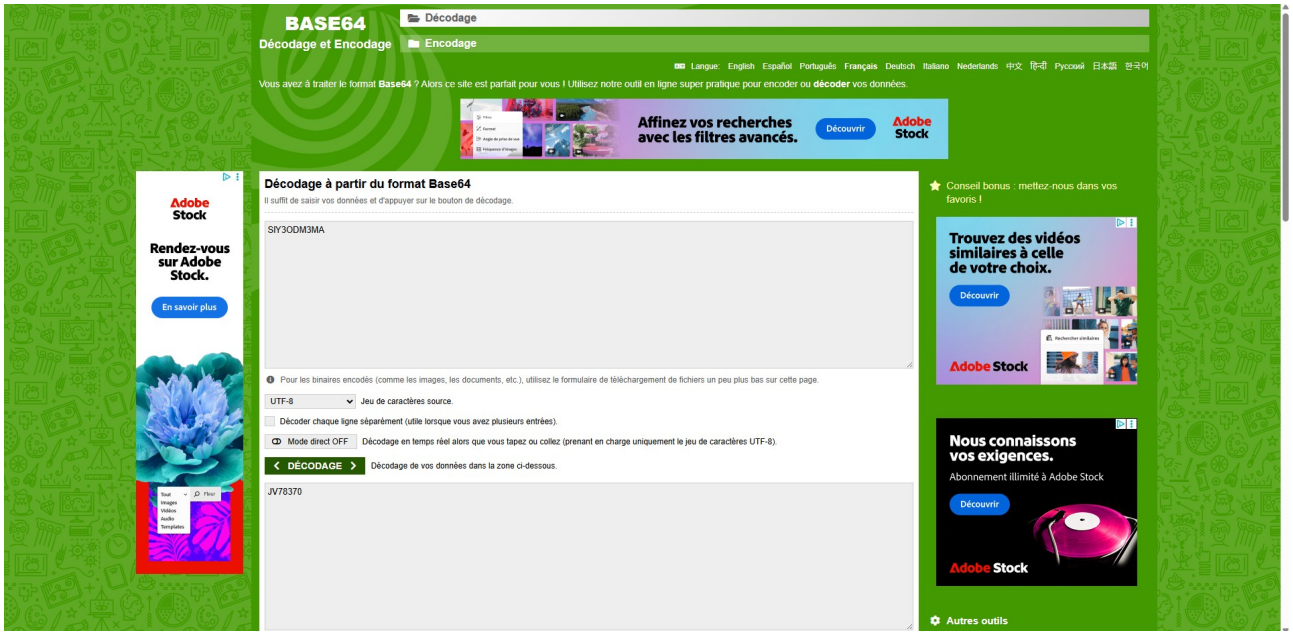
1. Copier la valeur :

   SlY3ODM3MA==

2. Coller cette valeur dans un site de décodage Base64.
3. Lancer le décodage.

Résultat obtenu :

JV78370



Décodage Base64 du mot de passe

9. Résultat de l'analyse

- **Utilisateur** : jed
- **Mot de passe** : JV78370

Les identifiants ont pu être récupérés simplement par analyse du trafic réseau.

10. Conclusion

Cette analyse montre que :

- L'authentification est réalisée via une requête **HTTP POST**.
- Le mot de passe est transmis dans le trafic réseau.
- Bien qu'encodé en Base64, le mot de passe reste **facilement récupérable**.

- L’utilisation du protocole **HTTP** représente une faille de sécurité importante.

La solution recommandée est l’utilisation de **HTTPS**, qui chiffre les échanges et empêche l’interception des identifiants.

Analyse d’une authentification HTTPS avec Wireshark

1. Objectif

L’objectif est d’analyser une authentification réalisée en **HTTPS** et de comparer avec le cas HTTP, afin de montrer que le chiffrement empêche la récupération des identifiants.

2. Mise en place de la connexion HTTPS

1. Ouvrir un navigateur web.
2. Accéder au NAS via HTTPS :

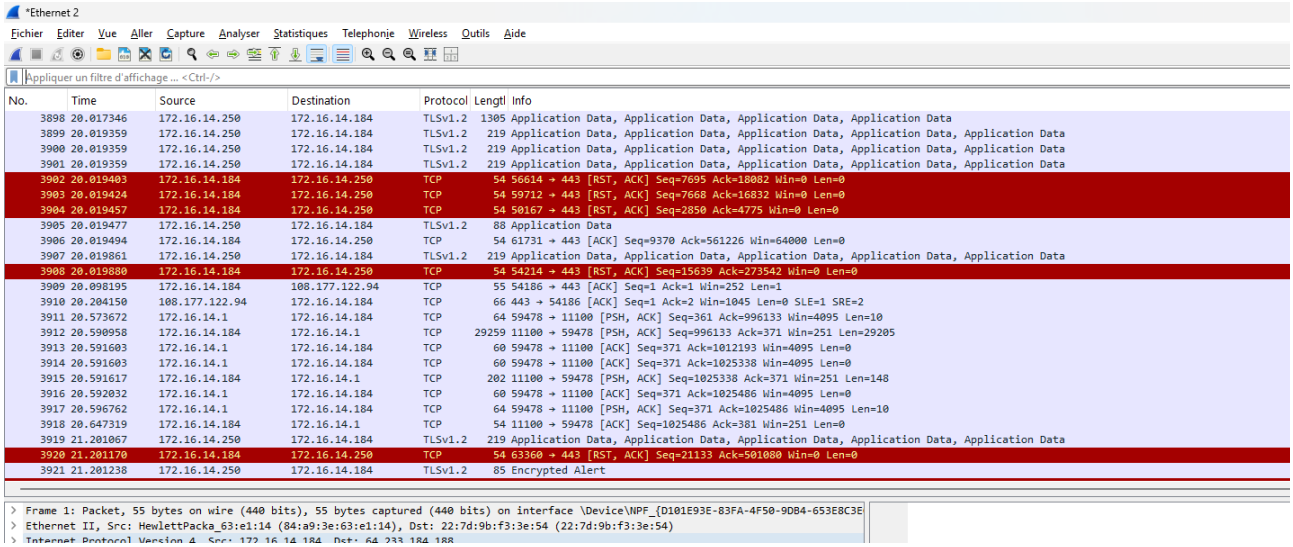
   https://172.16.14.250:8081

3. Accepter le certificat si nécessaire.
4. Se connecter avec un identifiant et un mot de passe.

Connexion au NAS en HTTPS

3. Capture du trafic réseau

1. Lancer Wireshark.
2. Sélectionner l’interface réseau.
3. Démarrer la capture.
4. Effectuer la connexion HTTPS au NAS.
5. Arrêter la capture.



Capture du trafic HTTPS dans Wireshark

4. Filtrage du trafic HTTPS

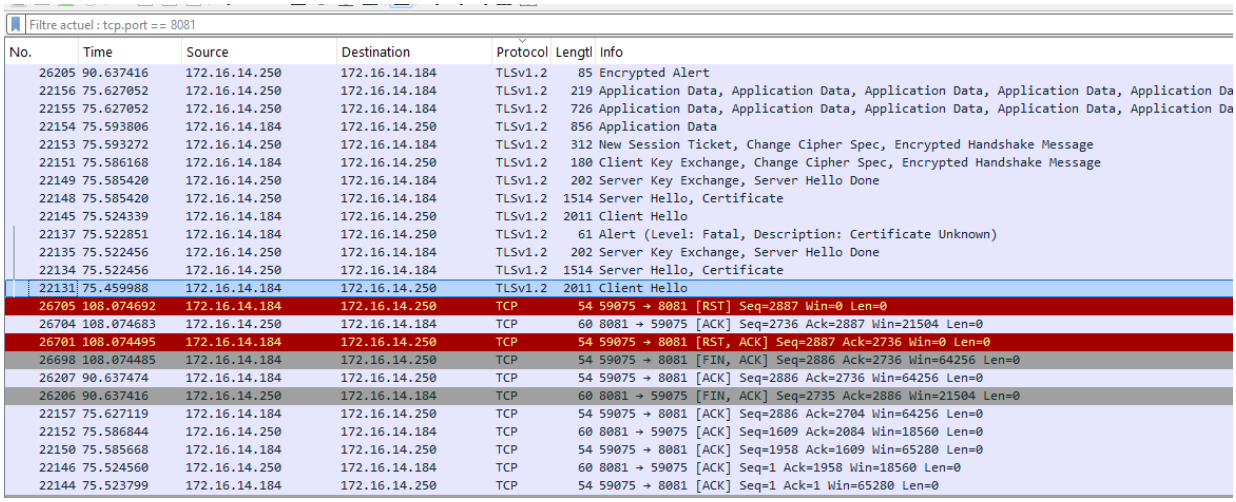
Appliquer le filtre suivant :

tls

ou :

tcp.port == 8081

On observe uniquement des trames TLS.



Filtrage des trames TLS

5. Analyse des échanges TLS

Contrairement au HTTP, on n'observe plus :

- de requêtes GET ou POST lisibles
- de paramètres user= ou pwd=
- de flux TCP exploitable

Les données applicatives apparaissent sous la forme :

Application Data

et sont **totalement chiffrées**.

22156	75.627052	172.16.14.250	172.16.14.184	TLSv1.2	219 Application Data, Application Data, Application Data, Application Data, Application Data
22155	75.627052	172.16.14.250	172.16.14.184	TLSv1.2	726 Application Data, Application Data, Application Data, Application Data, Application Data
22154	75.593806	172.16.14.184	172.16.14.250	TLSv1.2	856 Application Data

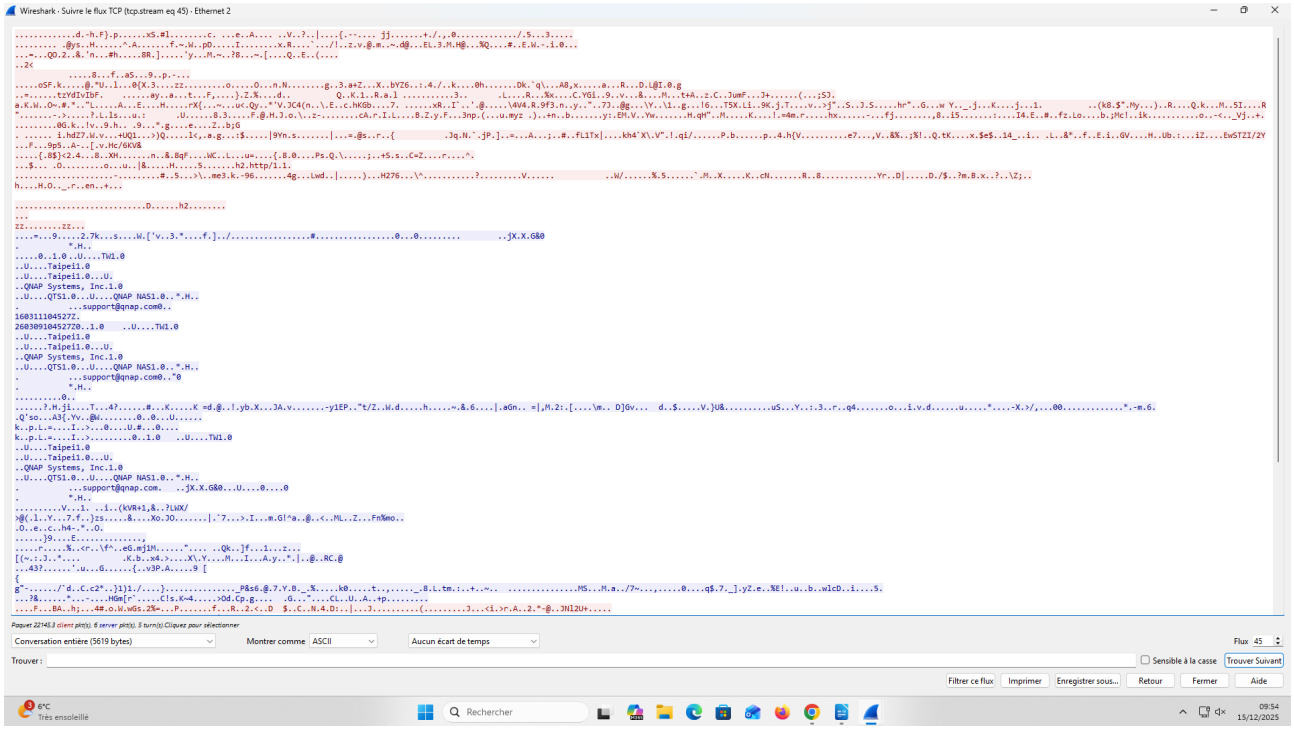
Contenu chiffré des trames TLS

6. Tentative d'analyse avec "Follow TCP Stream"

1. Clic droit sur une trame TLS
2. **Follow → TCP Stream**

Résultat :

- Données illisibles
- Aucun identifiant exploitable
- Aucun mot de passe visible



TCP Stream chiffré en HTTPS

7. Pourquoi il est très compliqué voire impossible de récupérer le mot de passe

Chiffrement TLS

- Les données sont chiffrées **avant** d'être envoyées sur le réseau
- Le mot de passe n'apparaît jamais en clair

Clés de chiffrement

- La clé de session est négociée lors du handshake TLS
- Elle n'est jamais transmise en clair

Certificat serveur

- Le client vérifie l'identité du serveur
- Empêche les attaques de type MITM simples

Limites de Wireshark

Sans :

- clé privée du serveur
- ou clés TLS du navigateur

Wireshark **ne peut pas déchiffrer** le trafic HTTPS.

8. Comparaison HTTP / HTTPS

Critère	HTTP	HTTPS
Données visibles	Oui	Non
Mot de passe récupérable	Oui	Non
Analyse avec TCP Stream	Possible	Impossible
Sécurité	Faible	Élevée

9. Conclusion

L'analyse montre que :

- En HTTP, les identifiants peuvent être interceptés et décodés.
- En HTTPS, le chiffrement TLS empêche toute récupération du mot de passe.
- L'utilisation de HTTPS est indispensable pour sécuriser les authentifications.

HTTPS protège efficacement contre l'analyse de trames réseau.

Lors de ce TP, certaines manipulations ont nécessité plusieurs essais, notamment pour la capture du trafic HTTPS.

Donc :

> Le chiffrement TLS rend l'analyse du trafic HTTPS inutilisable pour la récupération d'identifiants, contrairement au HTTP où les données sont visibles.

Bonus – Déchiffrement du trafic HTTPS à l'aide des clés TLS

Objectif du bonus

Ce bonus a pour objectif de montrer que, **dans des conditions très spécifiques et contrôlées**, il est possible d'analyser le contenu d'un trafic HTTPS.
Cette méthode dépasse le cadre d'une simple analyse réseau passive et nécessite un accès au poste client.

Principe général

Lors d'une connexion HTTPS, le navigateur génère des **clés de session TLS** qui servent à chiffrer et déchiffrer les données échangées avec le serveur.

Par défaut :

- ces clés restent **uniquement en mémoire**
- elles ne transitent **jamais sur le réseau**
- Wireshark ne peut donc pas les connaître

Cependant, il est possible de demander au navigateur de **journaliser ces clés TLS dans un fichier local**, à des fins de **débogage ou d'analyse**.

👉 Cette opération nécessite :

- un accès direct au poste client
- une action volontaire de l'utilisateur
- **aucune attaque réseau**

Mise en œuvre (niveau conceptuel)

1. Une variable d'environnement est définie sur le système afin d'indiquer au navigateur de sauvegarder les clés TLS dans un fichier.
2. Le navigateur est ensuite lancé **après** cette configuration.
3. Une connexion HTTPS est établie vers le NAS.

4. Wireshark est configuré pour utiliser ce fichier de clés TLS.
5. Le trafic HTTPS peut alors être **partiellement ou totalement déchiffré**.

Sans ce fichier de clés, le déchiffrement reste impossible.
– Fichier de clés TLS généré
– Paramétrage des clés TLS dans Wireshark

Résultat obtenu

Grâce aux clés TLS :

- les trames HTTPS ne sont plus affichées uniquement comme « Application Data »
- certaines requêtes HTTP internes (GET / POST) deviennent lisibles
- il devient alors possible d'observer les paramètres d'authentification

Limites et cadre de sécurité

Il est important de souligner que :

- cette méthode **ne fonctionne pas à distance**
- elle nécessite un **accès complet au poste client**
- elle ne correspond **pas à une attaque classique**
- elle ne remet **pas en cause la sécurité d'HTTPS**

Dans un contexte réel :

- un attaquant n'a généralement pas accès aux clés TLS du navigateur
- HTTPS reste donc un protocole sécurisé contre l'interception réseau passive

Conclusion du bonus

Ce bonus montre que :

- HTTPS protège efficacement les identifiants contre l'analyse réseau classique
- le déchiffrement n'est possible qu'en cas de **compromission ou contrôle du poste client**
- Wireshark seul ne permet pas de casser HTTPS

HTTPS reste indispensable pour la sécurisation des échanges réseau.